

# Entity Resolution Explained: Every Concept Behind the Amazon ML Challenge 2026 Solution

Sep 25, 2026 · @kushal

---

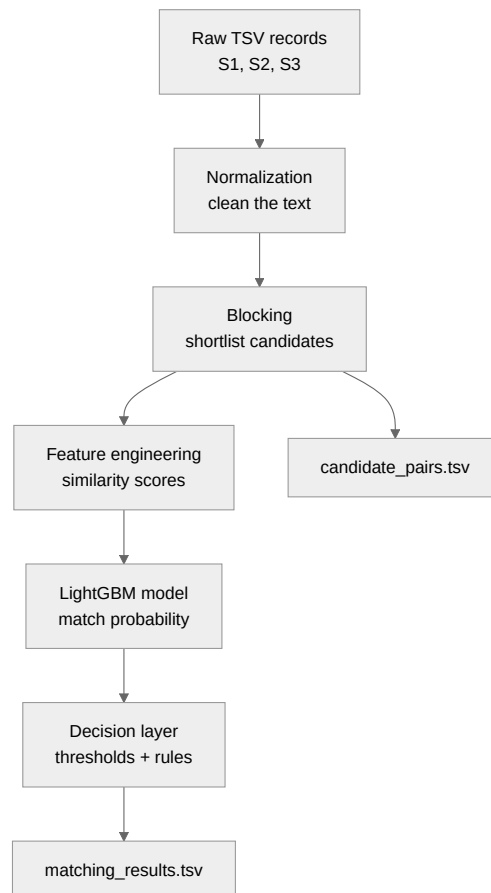
## Contents

1. How to read this document
2. 1. Machine learning foundations
3. 2. The problem: entity resolution
4. 3. Text normalization
5. 4. Blocking (candidate generation)
6. 5. Turning text into numbers
7. 6. String similarity algorithms
8. 7. Feature engineering
9. 8. The model: from decision trees to LightGBM
10. 9. Evaluation metrics
11. 10. Validation: measuring honestly
12. 11. The decision layer: from probabilities to answers
13. 12. Engineering concepts
14. 13. Glossary and interview preparation

# How to read this document

---

The solution is a five-stage pipeline: clean the text, shortlist likely pairs, measure how similar each pair is, let a model score each pair, then turn scores into final answers. Every concept in this document belongs to one of those stages.



The diagram reads top to bottom: each box is one stage, and the two files at the end are what the challenge asks you to submit.

Every concept below is explained with the same four questions:

1. **What is it?** A plain-language definition.
2. **Why does it exist?** The problem it was invented to solve.
3. **How does it work?** The mechanics, with a small example or formula.
4. **How do we use it here?** Where it appears in the notebook and why we chose it over alternatives.

No prior ML knowledge is assumed. Sections build on each other, so read them in order the first time. Later, use the glossary at the end as a quick reference before an interview.

# 1. Machine learning foundations

---

Machine learning means a computer learns a rule from examples instead of a human writing the rule by hand. In this challenge, the rule it learns is: "given two business records, are they the same business?"

## Why not just write rules by hand?

You could write: "if names are 90% similar and PIN codes match, it's a match." That works for easy cases. It breaks when a name has a typo *and* the PIN is missing *and* the address uses a landmark. Real data has thousands of such combinations. A human cannot write a rule for every one, but a model can learn the pattern from thousands of labeled examples.

## Supervised learning

**What:** learning from examples where the correct answer is known. Each example has inputs and a known answer, called the **label**.

**Why it exists:** if you have past examples with answers, the model can learn the mapping from input to answer and apply it to new cases.

**Here:** the training ground-truth file tells us which S2/S3 records truly match each S1 record. Every candidate pair becomes one example, labeled 1 (match) or 0 (not a match). The test set has no labels; the model must predict them.

## Classification

**What:** a supervised task where the answer is a category, not a number. **Binary classification** has two categories: yes or no.

**Here:** each pair is classified as match or non-match. The model actually outputs a **probability** between 0 and 1, such as 0.93, meaning "93% confident this is a match." We later decide where to draw the line (section 11).

## Features

**What:** the numeric inputs a model looks at. A model cannot read "Sharma Traders" directly; it needs numbers.

**Why they exist:** models do math on numbers. Features are how we translate raw data into something the model can compute with.

**Here:** for each pair we compute about 55 numbers, such as "name similarity = 0.91" and "postal codes agree = yes." Designing good features is called **feature engineering** (section 7).

| Term          | Meaning                  | Example in this project          |
|---------------|--------------------------|----------------------------------|
| Example (row) | One thing to predict     | One pair: S1-00001 with S2-00047 |
| Feature       | One numeric input        | Name similarity 0.91             |
| Label         | The known correct answer | 1 = same business                |
| Prediction    | The model's output       | Probability 0.93                 |

## Training vs inference

**Training** is when the model studies labeled examples and adjusts itself to make fewer mistakes. **Inference** is when the trained model makes predictions on new, unlabeled data. We train on the training set and run inference on the test set.

## Overfitting and generalization

**What: overfitting** means the model memorizes the training data, including its noise, instead of learning the true pattern. It scores perfectly on data it has seen and badly on new data.

**Generalization** is the opposite: performing well on data it has never seen.

**Analogy:** a student who memorizes past exam answers fails when the questions change. A student who understands the subject passes either way.

**Why it matters here:** the leaderboard uses hidden test data, including France, which the model never sees during training. A model that memorized US and India quirks would fail there. Many decisions in this project, such as GroupKFold (section 10) and no country feature (section 7), exist only to fight overfitting.

## 2. The problem: entity resolution

**Entity resolution (ER)** means figuring out which records, from different sources, describe the same real-world thing. It is also called record linkage, deduplication or entity matching.

### Why it exists

Companies collect data about the same businesses from many places: registrations, sellers, delivery partners, partner feeds. Each source spells names and addresses its own way, and there is no shared ID. Without ER, one business looks like three different businesses. That breaks fraud checks, analytics, payments and search.

### Key terms

| Term             | Meaning                                  | In this challenge                         |
|------------------|------------------------------------------|-------------------------------------------|
| Record           | One row of data from one source          | A row in train_source2.tsv                |
| Entity           | The real-world thing records describe    | The actual shop "Sharma Traders" in Delhi |
| Reference source | The clean, trusted list we match against | Source 1, already deduplicated            |
| Match            | Two records describing the same entity   | S1-00001 and S2-00047                     |
| Singleton        | An S1 entity with no matches anywhere    | The correct answer is an empty list       |
| One-to-many      | One S1 can match several S2/S3 records   | S1-00001 matches three records            |

### What "deduplicated reference source" means for us

Source 1 has no duplicates: each real business appears in it only once. This has a useful consequence. An S2 or S3 record can belong to **at most one** S1 entity, because a record describes one business, and that business appears once in S1. We exploit this later with one-to-one assignment (section 11). We also verify it in the EDA cell, which counts how many S2/S3 IDs are matched to more than one S1 in the ground truth.

### Why it's hard: the noise

| Noise type       | Example                           |
|------------------|-----------------------------------|
| Abbreviation     | "Pvt Ltd" vs "Private Limited"    |
| Typo             | "Maahlaxmi" vs "Mahalaxmi"        |
| Word order       | "Tech Gupta" vs "Gupta Tech"      |
| Trade name (DBA) | "Acme Corp dba QuickFix"          |
| Missing parts    | Address without a PIN code        |
| Landmark address | "Near SBI ATM, MG Road"           |
| Unseen language  | French names only in the test set |

## Two twin difficulties

ER has two opposite errors. Missing a true match makes one business look like two. Wrongly merging two different businesses makes two look like one. The challenge metric, F0.5, punishes the second error more (section 9), so the whole design leans toward being careful before declaring a match.

## EDA (exploratory data analysis)

**What:** looking at the data before modeling, to understand its shape. **Why:** assumptions that are wrong silently ruin a model. Our EDA cell checks the share of singletons, matches per entity, whether one-to-one holds, and whether matched pairs always share the same country label. Each answer changes a design decision.

### 3. Text normalization

**Normalization** means converting text into one standard form so that differences in formatting don't look like differences in meaning. It runs first, because every later stage compares text.

#### Why it exists

To a computer, "ABC Pvt. Ltd." and "abc private limited" are completely different strings. They share only a few characters in the same positions. Yet they are obviously the same name. If we compare raw strings, similarity scores get dragged down by capital letters, dots and abbreviations. Normalization removes that noise so similarity measures focus on real differences.

The general idea is called **canonicalization**: mapping many surface forms to one canonical (standard) form.

#### The steps, one by one

| Step                   | Before                | After                           | Problem it solves                    |
|------------------------|-----------------------|---------------------------------|--------------------------------------|
| Lowercase              | "SHARMA Traders"      | "sharma traders"                | Capital letters don't change meaning |
| Accent removal         | "Société Lumière"     | "societe lumiere"               | é vs e written inconsistently        |
| & to "and"             | "A & B"               | "a and b"                       | Two ways to write the same word      |
| Punctuation removal    | "Pvt. Ltd."           | "pvt ltd"                       | Dots and commas carry no identity    |
| Abbreviation expansion | "pvt ltd", "12 mg rd" | "private limited", "12 mg road" | Short and long forms become equal    |
| PIN joining            | "110 001"             | "110001"                        | Same postal code written two ways    |

#### Unicode and NFKD

**What:** Unicode is the standard that gives every character in every language a number. Some characters can be stored two ways: "é" as one character, or as "e" plus a separate accent mark.

**NFKD** (Normalization Form Compatibility Decomposition) is a rule that always splits characters into their base letter plus separate marks.

**Why we use it:** after NFKD, every "é" becomes "e" + accent. We then delete the accent marks, leaving plain "e". This matters because France appears only in the test set. Without this step, "Société" and "Societe" would look different even though they're the same word.

**Design choice:** we delete only accent marks, not all non-English characters. A cruder method ("keep ASCII only") would silently erase any Devanagari or other script entirely, turning names into empty strings.

#### Tokenization

**What:** splitting text into units called **tokens**, usually words. "sharma gupta tech" becomes the tokens ["sharma", "gupta", "tech"].

**Why:** many similarity measures work on sets of words, and blocking needs individual words to look up (section 4). We store each record's tokens as a set, so order doesn't matter.

## Regular expressions (regex)

**What:** a mini-language for describing text patterns. For example, `\b\d{6}\b` means "a standalone group of exactly six digits."

**Here:** we use regex to find postal codes (5–6 digits), house numbers, punctuation and whole-word abbreviations. The `\b` (word boundary) makes sure "st" is replaced only as a whole word, so the "st" inside "west" is never turned into "weststreet".

## Core name: removing legal suffixes

**What:** the core name is the business name with legal forms and filler words removed. "sharma traders private limited" becomes "sharma traders."

**Why:** legal forms (Inc, LLC, Pvt Ltd, SARL, SAS) are shared by thousands of businesses, so they carry almost no identity. Worse, the same business is often written with different suffixes in different sources. Removing them makes "Sharma Traders LLP" and "Sharma Traders Pvt Ltd" look identical, while "Sharma Traders" and "Gupta Traders" still look different.

**Safety net:** we keep both the full normalized name and the core name, and compute features on both. If stripping ever removes something important, the model can still rely on the full version.

## Extracted fields

| Field        | How it's found                       | Why it helps                                       |
|--------------|--------------------------------------|----------------------------------------------------|
| Postal code  | Last 5–6 digit number in the address | Very strong location evidence                      |
| All numbers  | Every digit group in the address     | House and shop numbers must agree                  |
| First number | First digit group                    | Usually the house number                           |
| Acronym      | First letter of each core word       | Matches "IBM" to "International Business Machines" |
| Trade name   | Text after "dba" or "trading as"     | Matches a business listed under its brand name     |

## What we deliberately did not do

We did not use geocoding (turning addresses into map coordinates) or look up business registries. Both would help, but the rules ban any external data lookup. Everything is learned from the provided files only.

## 4. Blocking (candidate generation)

**Blocking** is a cheap filtering step that shortlists plausible pairs before the expensive model runs. For each S1 record it keeps the 30-50 S2/S3 records that look roughly similar and discards the rest.

### The problem it solves: quadratic complexity

Comparing every record with every other record grows as the product of the two list sizes. Computer scientists call this  **$O(n \times m)$**  or **quadratic** complexity: double both lists and the work quadruples.

| S1 records | S2 + S3 records | All pairs  | At 1 million pairs/second |
|------------|-----------------|------------|---------------------------|
| 1,000      | 2,000           | 2 million  | 2 seconds                 |
| 100,000    | 200,000         | 20 billion | about 5.5 hours           |
| 1,000,000  | 2,000,000       | 2 trillion | about 23 days             |

With blocking at 40 candidates per S1, 100,000 S1 records need only 4 million pairs, which takes seconds. That's the whole reason blocking exists.

### Candidate generation

"Candidate generation" is the general name for this idea, and it appears far beyond ER. Search engines, recommendation systems and question-answering systems all first retrieve a shortlist cheaply, then rank it carefully. This two-stage pattern is often called **retrieve then rank**.

### The two numbers that judge a blocker

**Pair recall** (also called pair completeness) is the share of true matching pairs that survive blocking.

$$\text{Pair recall} = \frac{\text{true pairs kept by blocking}}{\text{all true pairs}}$$

This is the **recall ceiling** of the whole system. If blocking drops a true pair, the model never sees it, and nothing later can bring it back. That's why we aim for 99% or more.

**Reduction ratio** is the share of all possible pairs that blocking removed.

$$\text{Reduction ratio} = 1 - \frac{\text{candidate pairs}}{|S1| \times (|S2| + |S3|)}$$

A ratio of 0.99 means we compare only 1% of all pairs. The trade-off: a bigger shortlist raises recall but costs more time and gives the model more wrong candidates to reject.

**Oracle F0.5** is the score a perfect model would get on our candidates. It's the best score the system could possibly reach, so it tells you whether to improve blocking or the model.

## Top-K instead of a threshold

**What:** for each S1 record, keep the K most similar records (for example  $K = 15$ ), rather than "everything above similarity 0.7."

**Why:** data density varies. A common name like "Sharma Traders" might have 500 records above 0.7, while a rare name might have none. A threshold makes cost unpredictable and leaves some entities with nothing. Top-K guarantees each S1 gets a fixed, fair budget.

## Why we combine several blockers (union)

Each blocker has blind spots. Taking the **union** (all pairs any blocker found) covers each one's weakness with another's strength. This is the ensemble idea applied to recall.

| Blocker                  | Finds pairs where...                  | Misses pairs where...                  |
|--------------------------|---------------------------------------|----------------------------------------|
| TF-IDF on name           | Names are similar despite typos       | Name is a totally different trade name |
| TF-IDF on name + address | Address confirms a changed name       | Address missing and name changed       |
| Rare shared word         | A distinctive word appears in both    | Only common words are shared           |
| Same postal code         | Location identical, name different    | PIN missing or mistyped                |
| Embeddings (optional)    | Meaning or transliteration is similar | Very short or number-heavy text        |

The notebook prints each blocker's recall alone, which justifies keeping it. A blocker that adds no new true pairs can be dropped.

## Searching S2 and S3 separately

If we searched S2 and S3 together with  $K = 15$ , a crowded source could fill all 15 slots and push the true match from the other source out. Searching each separately gives each source its own K.

## Blocking within a country

Matches almost never cross countries, so we only search within the same country. It is faster and removes impossible pairs. If an S1 country has no records in the target source, we search everything instead. This keeps country an **open set**: nothing breaks when France appears. Switch `BLOCK_BY_COUNTRY` off if EDA shows matched pairs often have different country labels.

## Blocking methods we did not choose

| Method               | How it works                                    | Why not our main choice                  |
|----------------------|-------------------------------------------------|------------------------------------------|
| Standard blocking    | Group by an exact key, like first 3 letters     | One typo in the key loses the pair       |
| Sorted neighbourhood | Sort by a key, compare within a sliding window  | Sensitive to errors in the first letters |
| MinHash LSH          | Hash sets so similar ones collide               | Approximate, needs careful tuning        |
| Canopy clustering    | Cheap overlapping clusters, then compare inside | More parameters, similar result          |

Our approach is exact top-K similarity search, made fast through sparse matrix math (section 5). It's simple, exact and scales to GB-size data when done in chunks.

## 5. Turning text into numbers

To compare text with math, we turn each piece of text into a **vector**: a list of numbers. Similar texts should get similar vectors. This section explains how, from the simplest idea up to TF-IDF and embeddings.

### Vectors

**What:** an ordered list of numbers, like [0.2, 0, 0.7]. You can picture it as an arrow pointing in some direction in space. Two texts are similar when their arrows point in nearly the same direction.

### Bag of words

**What:** the simplest way to vectorize text. Make one column per word in the whole vocabulary, then count how many times each word appears in the text. Word order is ignored, like dumping words into a bag.

| Text                     | sharma | traders | gupta | limited |
|--------------------------|--------|---------|-------|---------|
| "sharma traders limited" | 1      | 1       | 0     | 1       |
| "gupta traders limited"  | 0      | 1       | 1     | 1       |

**Its problem:** these two rows share two of three words, so they look quite similar. But "traders" and "limited" are extremely common; the only word that identifies the business, "sharma" vs "gupta", is different. Counting treats all words as equally important. TF-IDF fixes this.

### N-grams

**What:** a sequence of N consecutive items. **Word n-grams** use words; **character n-grams** use letters. The character 3-grams of "sharma" are: sha, har, arm, rma.

**Why character n-grams matter here:** a typo breaks a whole word but only a few n-grams. "Maahlaxmi" and "Mahalaxmi" are different words, yet they share most of their 3-grams ("axm", "xmi", "lax"...), so they still look similar. Character n-grams also need no dictionary, so they work equally for Hindi transliterations and French names. We use `char_wb` n-grams, which are built within word boundaries and padded with spaces, so word starts and ends count too.

### TF-IDF

**What:** a way of weighting each term (word or n-gram) by how *informative* it is. It stands for **Term Frequency × Inverse Document Frequency**.

- **Term frequency (TF):** how often the term appears in this text. More often means more relevant to this text.
- **Inverse document frequency (IDF):** how rare the term is across all texts. A term that appears everywhere ("limited") gets a weight near zero. A rare term ("annapurna") gets a high weight.

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}$$

Here  $N$  is the total number of records and  $df(t)$  is the number of records containing term  $t$ . The log keeps very rare terms from getting absurdly large weights.

**Worked example:** with 100,000 records, "limited" appears in 40,000, so  $IDF = \log(100,000 / 40,000) \approx 0.9$ . "Annapurna" appears in 20, so  $IDF = \log(100,000 / 20) \approx 8.5$ . Sharing "annapurna" is about nine times stronger evidence than sharing "limited."

**sublinear\_tf:** we use  $1 + \log(\text{count})$  instead of the raw count, so a term repeated five times doesn't count five times as much.

**Why it exists:** it's the classic fix for bag-of-words' flaw. It was invented for search engines, to rank documents by how well rare query words match.

## Cosine similarity

**What:** a measure of how closely two vectors point in the same direction. It ranges from 0 (nothing in common) to 1 (identical direction).

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

$A \cdot B$  (the dot product) multiplies matching positions and adds them up.  $\|A\|$  is the vector's length. Dividing by lengths means a long address doesn't automatically look more similar than a short one.

**Shortcut we use:** scikit-learn's TF-IDF vectors are already scaled to length 1 (L2-normalized). Then cosine similarity is simply the dot product, which is much faster.

## Sparse matrices

**What:** a matrix where most values are zero, stored by remembering only the non-zero entries and their positions.

**Why:** the vocabulary of character  $n$ -grams can reach millions of columns, but a single name contains only about 30 of them. Storing all the zeros would need terabytes. Sparse storage needs only the few non-zero values.

**How we use it for blocking:** stack all  $S1$  vectors into matrix  $Q$  and all  $S2$  vectors into matrix  $T$ . Then one matrix multiplication  $Q \times T^T$  gives the cosine similarity of every  $S1$  record with every  $S2$  record. We do this in **chunks** of a few thousand  $S1$  rows at a time, keep only the top  $K$  per row, and discard the rest, so memory stays bounded.

## Inverted index

**What:** a lookup table from each word to the list of records containing it, like the index at the back of a book. It's the core data structure of every search engine.

**Why:** to find records sharing the word "annapurna", you jump straight to that word's list instead of scanning every record.

**How we use it:** the rare-token blocker builds this index for words appearing in at most 50 records (`max_df`). It pairs records sharing such words and scores each pair by the sum of IDF of shared words. Common words are excluded because their lists would be huge and useless.

## Embeddings

**What:** dense vectors, typically 384 or 768 numbers, produced by a neural network trained so that texts with similar *meaning* get similar vectors. Unlike TF-IDF, which matches spelling, embeddings can relate "Corporation" to "Company" or a transliterated name to its variant.

**Why they exist:** spelling-based methods cannot know that two different words mean the same thing. Neural networks trained on huge text collections learn such relationships.

**How we use them:** optionally, as one more blocker, using `paraphrase-multilingual-MiniLM-L12-v2`. It's a small (about 118 million parameter), multilingual, Apache-2.0-licensed sentence model, which fits the challenge's licence and size rules. Downloading a model is not an external data lookup: it contains no business registry information.

**Bi-encoder:** this model encodes each record separately into a vector, then compares vectors. That's fast enough for millions of records. A **cross-encoder**, which reads both records together, is more accurate but far slower (section 8).

## Nearest-neighbour search and FAISS

**What: k-nearest neighbours (kNN) search** means finding the K vectors most similar to a query vector. **FAISS** (Facebook AI Similarity Search) is a library that does this very fast on millions of dense vectors, on CPU or GPU.

**How we use it:** for the embedding blocker, FAISS finds the top K most similar S2/S3 records per S1 record. We use exact search ( `IndexFlatIP` , inner product), since vectors are normalized and inner product then equals cosine similarity. If FAISS isn't installed, the notebook falls back to chunked numpy matrix multiplication.

| Representation     | Captures                      | Speed             | Used for                 |
|--------------------|-------------------------------|-------------------|--------------------------|
| Char n-gram TF-IDF | Spelling, typos, any language | Fast (sparse)     | Main blockers + features |
| Word TF-IDF        | Shared informative words      | Fast (sparse)     | A feature                |
| Embeddings         | Meaning, transliteration      | Slower, GPU helps | Optional blocker         |

## 6. String similarity algorithms

Each string similarity measure is sensitive to a different kind of noise, so we compute many of them and let the model learn which to trust when. All of them output a score where higher means more similar. We compute them with **RapidFuzz**, a C++ library that is 10-100× faster than pure Python loops.

### Edit distance (Levenshtein)

**What:** the minimum number of single-character edits (insert, delete or substitute a letter) needed to turn one string into the other.

**Example:** "mahalaxmi" → "maahlaxmi" takes 2 edits (swapping two letters counts as two substitutions). "sharma" → "gupta" takes 5.

**Normalized similarity:** raw distance depends on length, so we scale it to 0-1:

$$\text{similarity} = 1 - \frac{\text{edit distance}}{\max(|a|, |b|)}$$

**Why it exists:** invented in 1965 for correcting errors in transmitted codes. It's the standard way to measure typos.

**Good at:** typos, missing or extra letters. **Bad at:** word reordering. "gupta tech" vs "tech gupta" looks very different, even though it's the same name.

`fuzz.ratio` is a close cousin that counts only insertions and deletions and gives a 0-100 score. We use both.

### Jaro-Winkler

**What:** a similarity score that counts matching characters appearing roughly in the same position, penalizes swapped characters, and gives a **bonus for matching prefixes** (the first up to 4 letters).

**Why it exists:** it was designed at the US Census Bureau specifically for matching people's names in records. People make fewer mistakes at the start of a name, so a shared beginning is strong evidence.

**Good at:** short strings and names with typos near the end. **Bad at:** long strings with shifted words.

### Token sort ratio

**What:** split both strings into words, sort the words alphabetically, rejoin them, then compute `ratio`.

**Example:** "tech gupta" and "gupta tech" both become "gupta tech", so the score is 100.

**Good at:** word-order transpositions, which the problem statement lists as a noise pattern.

## Token set ratio

**What:** compares the shared words separately from the words each string has extra, and takes the best of those comparisons. If one name's words are a subset of the other's, the score is 100.

**Example:** "sharma traders" vs "sharma traders private limited" scores 100, because every word of the first appears in the second.

**Good at:** extra words, like suffixes or "near SBI ATM" added to an address. **Risk:** "sharma" vs "sharma traders gupta foods" also scores 100, so it must be combined with other features. That's the model's job.

## Partial ratio

**What:** finds the best-matching substring of the longer string with the length of the shorter one, then scores that alignment.

**Example:** "apex motors" vs "apex motors of boston inc" scores high, because "apex motors" appears inside.

**Good at:** truncated names and short forms embedded in long ones.

## Jaccard similarity

**What:** the size of the overlap divided by the size of the union of two sets.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

**Example:** {sharma, traders} vs {sharma, textiles}: overlap = 1 word, union = 3 words, so  $J = 0.33$ .

**Here:** we compute Jaccard on name words, address words, and the set of numbers in the address.

## IDF-weighted overlap

**What:** Jaccard where each word counts by its IDF weight (section 5) instead of counting as 1.

$$\text{IDF overlap}(A, B) = \frac{\sum_{t \in A \cap B} \text{idf}(t)}{\sum_{t \in A \cup B} \text{idf}(t)}$$

**Why:** plain Jaccard says {sharma, traders} and {gupta, traders} overlap by 33%. But the shared word "traders" is common and the differing words are the identifying ones. With IDF weighting the overlap drops close to 0, which correctly signals "probably different businesses."

## Which measure catches which noise

| Noise              | Best measures                               |
|--------------------|---------------------------------------------|
| Typo               | Levenshtein, Jaro-Winkler, character TF-IDF |
| Word order swapped | Token sort ratio, token set ratio, Jaccard  |

| Noise                      | Best measures                          |
|----------------------------|----------------------------------------|
| Extra words or suffixes    | Token set ratio, core-name comparisons |
| Truncated or embedded name | Partial ratio                          |
| Common vs rare words       | IDF-weighted overlap, TF-IDF cosine    |
| Abbreviation (IBM)         | Acronym match                          |

No single measure handles all of these, which is exactly why we feed them all to a model instead of choosing one.

## 7. Feature engineering

**Feature engineering** is designing the numeric inputs the model learns from. For tabular problems like this one, good features usually matter more than the choice of model. We compute about 55 features per pair, in five groups.

### Why hand-designed features instead of letting a model read raw text?

Deep learning models can learn features themselves, but they need far more labeled data, GPU time and tuning. With three days and tabular data, well-chosen similarity features plus gradient boosting is faster, more reliable and easier to explain. Each feature also encodes human knowledge the model would otherwise have to discover.

### The five feature groups

| Group                     | Examples                                                                                     | What it tells the model                                 |
|---------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Name similarity           | ratio, Jaro-Winkler, token set/sort, partial, Levenshtein on full and core name              | How alike the names are, under different kinds of noise |
| Vector similarity         | TF-IDF cosine on core name, name words, address, name + address                              | Weighted overall similarity                             |
| Overlap and exact signals | Jaccard, IDF overlap, core-name equal, first word equal, acronym match                       | Whether distinctive parts coincide                      |
| Address and location      | Address token set, postal agreement, house-number agreement, number Jaccard, address missing | Whether they're at the same place                       |
| Context and meta          | Rank, gap, reverse rank, mutual best, candidate counts, blocker scores, source is S3         | How this pair compares to its competitors               |

### Tri-state (three-valued) features

**What:** a feature with three values instead of two: **+1** both present and equal, **-1** both present and different, **0** at least one missing.

**Why:** "different PIN code" is strong evidence *against* a match. "PIN code missing" is *no evidence at all*. A simple yes/no "PIN matches" feature would treat missing as no, and would wrongly punish the many true matches whose address has no PIN. The problem statement explicitly warns that components are often missing.

**Here:** used for postal code ( `postal_agree` ) and house number ( `fnum_agree` ).

### Context features: the key idea

**The problem:** a raw similarity score is ambiguous. A name similarity of 0.8 could be the clear winner among an S1's candidates, or one of ten look-alikes that all score 0.8. The first is probably a match; the second probably isn't.

**What context features do:** they describe a pair *relative to its competitors* instead of in isolation.

| Feature                            | Meaning                                                                      | Why it helps                             |
|------------------------------------|------------------------------------------------------------------------------|------------------------------------------|
| Rank ( <code>_rk</code> )          | Position of this candidate among this S1's candidates (1 = most similar)     | True matches are usually near the top    |
| Gap ( <code>_gap</code> )          | Best candidate's score minus this one's                                      | A big lead means confidence              |
| Reverse rank ( <code>_rrk</code> ) | Rank of this S1 among all S1s competing for this candidate                   | The candidate may fit another S1 better  |
| Mutual best                        | Each is the other's top choice                                               | A very strong match signal               |
| Candidate counts                   | How many candidates this S1 has, and how many S1s this candidate appears for | Crowded neighbourhoods need more caution |

This idea is how the model separates near-duplicates like "Sharma Traders, Delhi" and "Sharma Traders, Mumbai." Context features are computed within each S1's candidate group, which is also why validation must keep groups together (section 10).

## Blocker scores as features

The score and rank from each blocker are kept as features too. A pair found by four blockers at rank 1 is more convincing than one found by a single blocker at rank 14. Pairs a blocker didn't find get score 0 and rank 99.

## Domain shift and why there is no country feature

**Domain shift** (also called distribution shift) means the test data differs from the training data in some systematic way. Here the shift is concrete: France exists only in the test set.

**The risk:** if "country = India" were a feature, the model might learn rules like "for India, a lower name similarity is fine." For France, a category it never saw, its behaviour would be unpredictable.

**Our choice:** no country feature at all. Similarity features mean the same thing in every country: a name similarity of 0.95 is strong evidence whether the name is French, Indian or American. This is a standard technique for **out-of-distribution generalization**: rely on features whose meaning doesn't depend on the domain.

Country is still stored with each pair, but only for the leave-one-country-out evaluation (section 10), never as a model input. The `same_country` feature is fine because it's a comparison, not a category.

## Labels

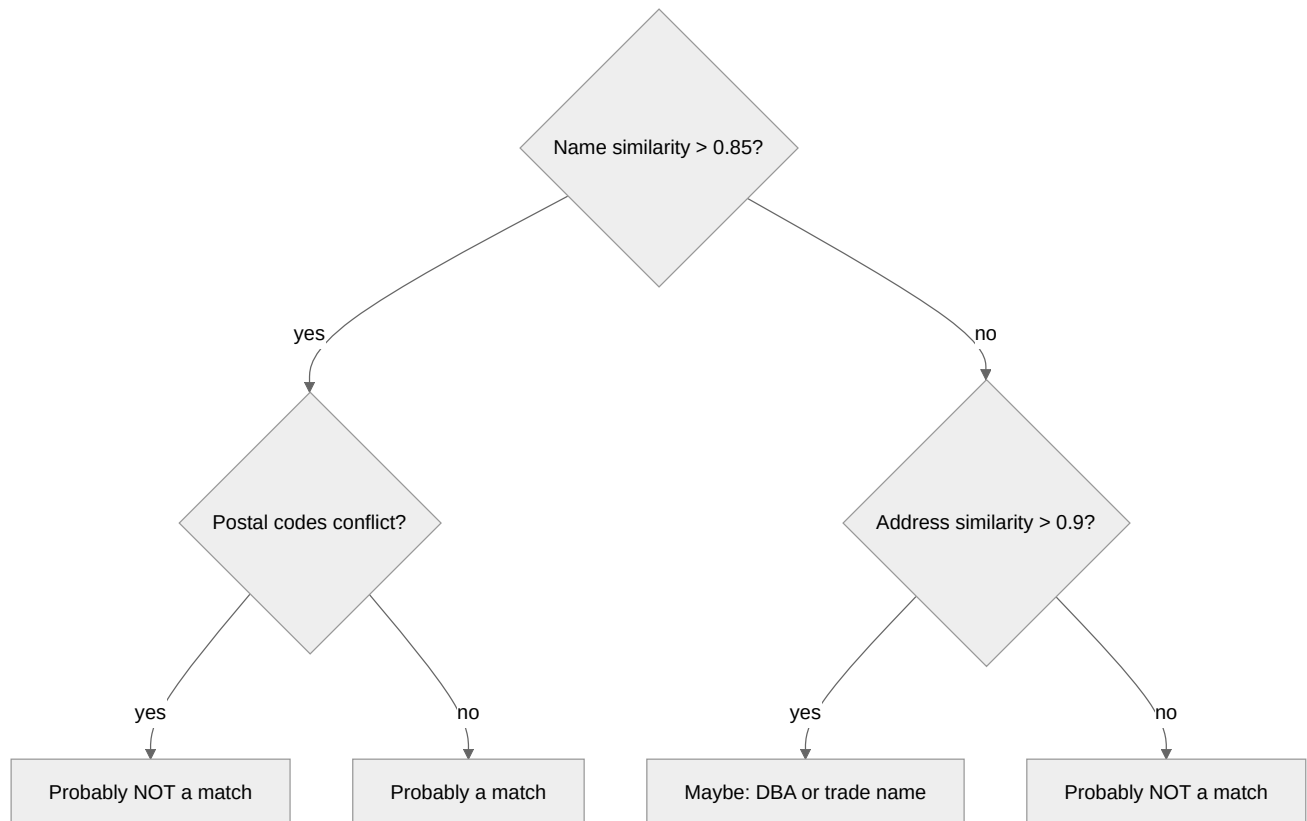
Each candidate pair is labeled 1 if it appears in the ground truth, else 0. True pairs lost by blocking never get a row, which is exactly why blocking recall caps the system.

## 8. The model: from decision trees to LightGBM

The matching model is **LightGBM**, a gradient-boosted decision tree library. To understand it, build up in three steps: one decision tree, many trees together (ensembles), and trees that correct each other (boosting).

### Decision tree

**What:** a flowchart of yes/no questions about the features, ending in a prediction. The algorithm picks, at each step, the question that best separates matches from non-matches in the training data.



The tree above learns an **interaction**: high name similarity means a match *unless* the postal codes conflict. Each end box is a **leaf**, holding the prediction for all pairs that reach it.

**How it picks questions:** it tries many features and thresholds, and chooses the split that makes the two resulting groups the most "pure" (mostly matches on one side, mostly non-matches on the other).

**Weakness:** one deep tree memorizes training data (overfits); one shallow tree is too crude. The fix is to combine many trees.

### Ensembles: bagging vs boosting

An **ensemble** combines many models so their errors cancel out, much like averaging many people's guesses beats most individual guesses.

|                                  | Bagging (e.g. random forest)                                | Boosting (e.g. LightGBM, XGBoost)                                        |
|----------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------|
| How trees are built              | Independently, in parallel, each on a random sample of data | One after another; each new tree focuses on the previous trees' mistakes |
| Main effect                      | Reduces variance (random noise in predictions)              | Reduces bias (systematic errors) and variance                            |
| Typical accuracy on tabular data | Good                                                        | Usually best                                                             |
| Risk                             | Low                                                         | Overfits if trained too long, hence early stopping                       |

## Gradient boosting

**What:** start with a simple guess (e.g. the overall match rate). Measure the error on every training example. Train a small tree to predict that error. Add it to the model, scaled down by a **learning rate**. Repeat hundreds of times. Each tree nudges predictions toward the truth.

**Why "gradient":** the "error" each new tree fits is the gradient (the direction of steepest improvement) of the **loss function**, a formula measuring how wrong the predictions are. For yes/no problems the loss is **log loss** (binary cross-entropy), which heavily punishes confident wrong answers.

$$\text{LogLoss} = -[y \log(p) + (1 - y) \log(1 - p)]$$

Here  $y$  is the true label (0 or 1) and  $p$  is the predicted probability. Predicting  $p = 0.99$  for a true non-match ( $y = 0$ ) costs about 4.6; predicting 0.6 costs about 0.9.

**Analogy:** a golfer's first shot lands near the hole; each next shot corrects the remaining distance. The learning rate is taking smaller, careful shots so you don't overshoot.

## Why LightGBM specifically

**LightGBM** is Microsoft's gradient boosting library. Two inventions make it fast:

- **Histogram binning:** it groups each feature's values into about 255 buckets, so finding the best split checks 255 options instead of millions.
- **Leaf-wise growth:** it always splits the leaf that reduces error the most, rather than growing the tree level by level. That gives more accuracy per tree.

| Reason                             | Why it matters here                                           |
|------------------------------------|---------------------------------------------------------------|
| Best model family for tabular data | Our input is ~55 numeric similarity scores                    |
| Learns interactions automatically  | "High name similarity but PIN conflict" needs no manual rule  |
| No feature scaling needed          | Scores on 0-1, 0-100 and ranks can be mixed freely            |
| Handles missing values natively    | Missing addresses don't need special handling                 |
| Fast on CPU                        | Millions of pairs train in minutes, fitting a 3-day challenge |
| Interpretable                      | Feature importance shows which signals matter                 |
| MIT licence, tiny size             | Fits the challenge's licence and 8B-parameter rules easily    |

## Hyperparameters we set

**Hyperparameters** are settings chosen before training, unlike parameters (the tree splits) which the model learns.

| Hyperparameter          | Value      | What it controls                                                             |
|-------------------------|------------|------------------------------------------------------------------------------|
| learning_rate           | 0.05       | How big each tree's correction is; smaller = slower but more careful         |
| num_leaves              | 63         | Maximum leaves per tree; more = more complex patterns, more overfitting risk |
| min_child_samples       | 40         | A leaf needs at least 40 examples, so no rule is based on a handful of pairs |
| feature_fraction        | 0.8        | Each tree sees a random 80% of features, adding diversity                    |
| bagging_fraction + freq | 0.8, 1     | Each tree trains on a random 80% of rows                                     |
| lambda_l2               | 1.0        | L2 regularization: penalizes extreme leaf values to reduce overfitting       |
| num_boost_round         | up to 5000 | The ceiling on number of trees; early stopping picks the real number         |

These are sensible defaults, not tuned values. **Regularization** is the general name for any technique that restrains a model from overfitting.

## Early stopping

**What:** train while watching the score on validation data; stop when it hasn't improved for 150 rounds, and keep the best round.

**Why:** boosting keeps reducing training error forever, but after some point it starts memorizing noise and validation error rises. Early stopping finds that point automatically, so we don't have to guess the number of trees.

## Class imbalance and hard negatives

**Class imbalance** means one class is much rarer than the other. Here, among ~40 candidates per S1, usually only 0–3 are true matches, so positives may be only 2–5% of rows.

**Common fixes** are reweighting the rare class or resampling. **We don't use them**, because we never use the model's default 0.5 cutoff. Instead we choose thresholds that directly maximize the metric (section 11). Reweighting would only shift probabilities, which threshold tuning already handles.

**Hard negatives** are non-matches that look very similar to matches. Because our negatives come from the blocker, they're all hard negatives: the most similar non-matching records. That's ideal training material, since these are exactly the cases the model must learn to reject. Random negatives ("Sharma Traders" vs "Apex Motors") would teach it nothing.

## AUC as the training monitor

During training we watch **AUC** (section 9) on validation data. It measures ranking quality independent of any threshold, which suits the stage where we don't yet know the threshold.

## Feature importance

**Gain importance** adds up how much each feature reduced the loss across all its splits. The notebook prints the top 25. It helps you explain the model, spot useless features, and catch bugs: a feature that's suspiciously dominant may leak the label.

## Fold averaging at test time

Cross-validation trains five models (section 10). For the test set we average all five models' predictions instead of retraining one. This is a form of bagging: averaging models trained on slightly different data reduces variance, and it costs no extra training time.

## Alternatives we considered

| Model                                 | Strength                                       | Why not the main model                                                                     |
|---------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------|
| Logistic regression                   | Simple, fast, interpretable baseline           | Linear: can't learn interactions without manual feature crosses                            |
| Random forest                         | Robust, little tuning                          | Usually slightly less accurate than boosting                                               |
| XGBoost / CatBoost                    | Equally strong boosting libraries              | LightGBM is faster on large data; could be added as an ensemble                            |
| Siamese neural network                | Learns its own representations                 | Needs much more data and GPU time                                                          |
| Cross-encoder (e.g. XLM-RoBERTa, MIT) | Reads both records together; best at semantics | Slow on millions of pairs; best used as an extra feature for LightGBM                      |
| LLM ( $\leq 8B$ , MIT/Apache)         | Understands context and language               | Far too slow and costly for millions of pairs; possible re-ranker for a few ambiguous ones |

**Cross-encoder vs bi-encoder:** a bi-encoder (section 5) turns each record into a vector separately, which is fast. A cross-encoder feeds both records into the network at once, so it can compare word by word. It's more accurate but must run once per pair, which is why it belongs after blocking, not in it.

## 9. Evaluation metrics

The challenge scores submissions with **macro-averaged F0.5 per S1 entity**. To understand that phrase, build up from the confusion matrix.

### Confusion matrix

**What:** a table counting the four possible outcomes of a yes/no prediction.

|                    | Truly a match                    | Truly not a match                     |
|--------------------|----------------------------------|---------------------------------------|
| Predicted match    | True positive (TP): correct link | False positive (FP): false merge      |
| Predicted no match | False negative (FN): missed link | True negative (TN): correctly ignored |

Every metric below is a different way of combining these four counts.

### Why not accuracy?

**Accuracy** = (TP + TN) / everything. It's misleading when one class is rare. If 97% of candidate pairs are non-matches, a model that always says "no match" gets 97% accuracy while finding nothing. This is why ER never uses accuracy.

### Precision

**What:** of the matches you predicted, how many were right?

$$\text{Precision} = \frac{TP}{TP + FP}$$

**Plain meaning:** "when I say match, how often am I correct?" Low precision means many false merges.

### Recall

**What:** of the true matches that exist, how many did you find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

**Plain meaning:** "how many real links did I catch?" Low recall means many missed links.

### The trade-off

Precision and recall pull against each other. Predicting more matches raises recall but brings more false positives, lowering precision. Predicting fewer does the opposite. The **threshold** (section 11) is the dial that moves between them.

### F1 and F-beta

**F1** combines precision and recall into one number using the **harmonic mean**, which stays low if either one is low. You can't cheat it by maximizing only one.

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

**F-beta** generalizes this with a weight  $\beta$  that sets how much recall matters relative to precision:

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R}$$

$\beta = 1$  gives F1 (equal weight).  $\beta = 2$  favours recall.  **$\beta = 0.5$  favours precision.** With  $\beta = 0.5$ , the formula becomes  $1.25 \cdot P \cdot R / (0.25 \cdot P + R)$ , exactly as in the problem statement.

**Why F0.5 here:** in real ER, merging two different businesses (a false positive) is worse than missing a link (a false negative). A false merge can mix up payments, reviews or fraud records of two separate companies. A missed link just leaves a duplicate.

| Precision | Recall | F1   | F0.5 |
|-----------|--------|------|------|
| 1.00      | 0.50   | 0.67 | 0.83 |
| 0.50      | 1.00   | 0.67 | 0.56 |
| 0.90      | 0.90   | 0.90 | 0.90 |

The first two rows have the same F1, but F0.5 clearly rewards the careful model (first row) over the greedy one (second row).

## Macro vs micro averaging

**Micro averaging:** add up TP, FP and FN over all entities, then compute one F-score. Big entities with many matches dominate.

**Macro averaging:** compute F-score separately for each S1 entity, then take the plain average. Every entity counts equally, whether it has 5 matches or none.

**The challenge uses macro.** Consequence: an S1 with one match counts just as much as an S1 with ten. Getting many small entities fully right matters more than being very good on a few large ones.

## Singletons: the hidden 1.0 or 0.0

For an S1 entity with no true matches, precision and recall aren't defined in the usual way, so the challenge defines them:

| Truth       | Prediction | Score |
|-------------|------------|-------|
| No matches  | Empty list | 1.0   |
| No matches  | Anything   | 0.0   |
| Has matches | Empty list | 0.0   |

This is extremely asymmetric. One wrong match on a singleton wipes out a full point. If 30% of entities are singletons, they are 30% of the whole score. That's why we have a dedicated singleton rule (section 11).

## Worked example (from the problem statement)

Prediction for S1-00001: [S2-00047, S2-00193, S3-00812]. Truth: [S2-00047, S3-00812].

- $TP = 2$  (S2-00047, S3-00812),  $FP = 1$  (S2-00193),  $FN = 0$
- $Precision = 2/3 = 0.667$ ,  $Recall = 2/2 = 1.0$
- $F0.5 = 1.25 \times 0.667 \times 1.0 / (0.25 \times 0.667 + 1.0) = 0.714$

Dropping S2-00193 would score 1.0. One extra wrong ID cost 0.29 on this entity.

## AUC (area under the ROC curve)

**What:** the probability that a randomly chosen true match gets a higher score than a randomly chosen non-match. 1.0 is perfect ranking; 0.5 is random guessing.

**The ROC curve** plots the true positive rate against the false positive rate at every possible threshold. AUC is the area under that curve.

**Why we use it:** only to monitor training and early stopping, because it judges ranking quality without choosing a threshold. It's *not* the challenge metric. A model can have excellent AUC and still score poorly if the threshold is badly chosen.

## Metrics for each stage

| Stage          | Metric          | Question it answers                            |
|----------------|-----------------|------------------------------------------------|
| Blocking       | Pair recall     | Did the true matches survive the shortlist?    |
| Blocking       | Reduction ratio | How much work did we save?                     |
| Blocking       | Oracle F0.5     | What's the best score possible from here?      |
| Model training | AUC, log loss   | Does the model rank matches above non-matches? |
| Final output   | Macro F0.5      | The official score                             |

The notebook implements the exact challenge metric ( `f05_entity` and `macro_f05` ) plus a fast vectorized version ( `fast_score` ) used during threshold search. It checks that both agree.

## 10. Validation: measuring honestly

**Validation** means estimating how well the model will do on data it has never seen, *before* spending a leaderboard submission. The test set has no labels and submissions are limited to 5 per day, so an honest local score is essential.

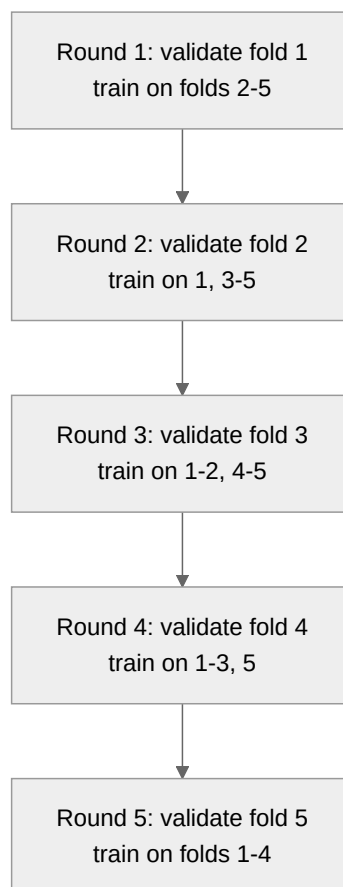
### Train, validation and test sets

| Set        | Has labels?                    | Used for                                    |
|------------|--------------------------------|---------------------------------------------|
| Training   | Yes                            | The model learns from it                    |
| Validation | Yes, but hidden from the model | Measuring performance and choosing settings |
| Test       | No                             | Final predictions for submission            |

**Golden rule:** a model must never be scored on data it trained on. It would look far better than it really is, just as a student who has seen the exam questions.

### K-fold cross-validation

**What:** split the labeled data into K equal parts (**folds**; we use  $K = 5$ ). Train on 4 folds and validate on the fifth. Rotate so each fold is the validation set once. You get 5 models and 5 scores.



**Why instead of one split:** a single split wastes data and gives a noisy score, since you might get lucky or unlucky with which rows land in validation. Cross-validation uses every row for validation exactly once and averages out the luck.

## Data leakage

**What:** information from the validation data accidentally reaching the model during training. The model then looks better in validation than it will be on the real test set.

**Why it's dangerous:** you can't see it. The score looks great, you submit, and the leaderboard score is much lower. Leakage is one of the most common and costly mistakes in ML competitions.

## GroupKFold: why groups = S1 entity

**The problem with plain K-fold here:** each S1 entity has about 40 candidate pairs. Plain K-fold shuffles *rows*, so some of an entity's pairs land in training and others in validation. Two leaks follow:

1. **Shared context:** our context features (rank, gap) are computed from all candidates of an S1. If some of those candidates are in training, the model has effectively seen part of the validation entity's situation.
2. **Shared record:** the S1 record itself appears in both sets, so the model can memorize things about that specific record.

**GroupKFold** keeps every row of a group in the same fold. With groups = S1 entity, each validation entity is completely unseen, exactly as at test time, where every S1 is new.

**General lesson for interviews:** whenever rows are related (same user, same patient, same entity, same time period), split by the group, not by row.

## Out-of-fold (OOF) predictions

**What:** each training row's prediction made by the one fold model that did *not* train on it. After 5 rounds, every row has exactly one honest prediction.

**Why we need them:** we tune thresholds (section 11) on predictions. If we tuned on predictions from models that trained on those same rows, they'd be over-confident, and we'd choose thresholds that are too loose. OOF predictions behave like test predictions, so thresholds tuned on them transfer to the test set.

**Bonus:** the OOF macro F0.5 is our main local score, computed over *all* S1 entities, including those with no candidates.

## Leave-one-country-out (LOCO)

**What:** train on every country except one, then validate on the held-out country. Here: train on US, test on India; then train on India, test on US.

**Why:** France appears only in the test set. Normal cross-validation mixes US and India in both training and validation, so it can't tell us how the model handles an unseen country. LOCO simulates exactly that situation.

**How to read it:** if LOCO scores are close to the OOF score, the model generalizes across countries. A big drop means it relies on country-specific quirks and will likely struggle on France. This is a form of **domain generalization** testing.

## Public vs private leaderboard

**What:** the public leaderboard scores part of the test set during the challenge; the private leaderboard scores the rest after it ends. The final ranking uses the private one.

**Why it exists:** to stop **leaderboard overfitting**. If you keep tweaking until the public score rises, you may be fitting noise in that particular subset. The private score then drops.

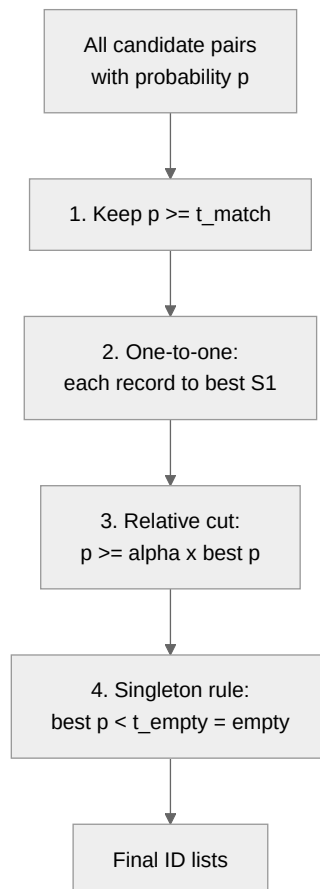
**Our defence:** choose the final submission by local cross-validation *and* LOCO, and use the public leaderboard only as a sanity check. When they disagree, trust the validation that has more data and simulates France.

## Random seeds and reproducibility

**A random seed** fixes the random choices (fold shuffling, row sampling in trees), so re-running gives identical results. We set `SEED = 42`. The challenge may ask you to reproduce your results from the code, so this matters.

## 11. The decision layer: from probabilities to answers

The model outputs a probability for each pair, but the submission needs a list of IDs per S1. The **decision layer** converts one into the other with four rules, tuned directly for macro F0.5. This stage often adds more score than model tweaks.



### Why not just "probability > 0.5"?

0.5 is the cutoff that maximizes **accuracy** when probabilities are well calibrated. Our metric is different in three ways: it favours precision ( $\beta = 0.5$ ), it's averaged per entity, and singletons give 1.0 or 0.0. The best cutoff for that metric is usually higher than 0.5, and the only reliable way to find it is to try values and measure.

### Rule 1: match threshold ( $t_{\text{match}}$ )

**What:** keep only pairs whose probability is at least  $t_{\text{match}}$ .

**Why:** it's the main precision-recall dial. Higher  $t_{\text{match}}$  means fewer, surer matches: more precision, less recall. Because F0.5 favours precision, the tuned value tends to be on the strict side.

### Rule 2: one-to-one assignment

**What:** if an S2/S3 record passes the threshold for several S1 entities, keep it only for the S1 where its probability is highest.

**Why it's valid:** Source 1 is deduplicated, so each S2/S3 record describes a business that appears exactly once in S1. It can't truly match two S1 entities (section 2). Any extra claims are guaranteed false positives. The EDA cell verifies this assumption on the training labels.

**Why it helps so much:** near-duplicates like "Sharma Traders, Delhi" and "Sharma Traders, Delhi (Branch 2)" might both claim the same S2 record. Without this rule, one of them always gets a false positive, which can turn a singleton's 1.0 into 0.0.

**The concept:** this is a **bipartite matching** problem: two sets (S1 and S2/S3), with edges between them weighted by probability. The famous exact solver is the **Hungarian algorithm**, but it forces one-to-one on *both* sides. That's wrong here, because one S1 can legitimately have several matches. Our greedy rule ("each record goes to its best S1") enforces the constraint only on the side where it's true, and runs in one sort.

### Rule 3: relative cut (alpha)

**What:** within each S1, keep a candidate only if its probability is at least  $\alpha \times$  the best candidate's probability.

**Why:** if the top candidate scores 0.98 and another scores 0.55, the second is much less certain than the first, even if it passes `t_match`. The relative cut drops such weak tag-alongs.  $\alpha = 0$  disables the rule; the search decides whether it helps.

### Rule 4: singleton rule (`t_empty`)

**What:** if an S1's best remaining candidate scores below `t_empty`, predict an empty list for that S1.

**Why:** it targets the singleton asymmetry directly (section 9). Predicting empty earns 1.0 on a true singleton, while one weak guess would score 0.0. Separating `t_empty` from `t_match` lets the search find, for example, "accept extra matches above 0.6, but only if the entity has at least one match above 0.8."

### Unseen-country margin

**What:** for S1 records from a country not seen in training (France), both thresholds are raised by `UNSEEN_COUNTRY_MARGIN` (0.05 by default).

**Why:** the model has never seen French data, so its probabilities there are less trustworthy. Under a precision-heavy metric, being more careful under uncertainty is rational. It's written generically (any unseen country), not hard-coded to France, which respects the open-set rule. This is a judgment call, so present it as one and tune it using LOCO results.

### Tuning: grid search

**What:** try every combination of values from a list for each parameter, score each combination, and keep the best.

**Why not gradient-based optimization:** the metric is a **non-differentiable** step function: moving a threshold slightly changes nothing until a pair crosses it, then the score jumps. Gradients don't exist for such functions, but with only four parameters trying combinations is cheap.

**Two-stage (coordinate) search:** a full grid of all four parameters would take thousands of evaluations. Instead we search `t_match` and one-to-one first (with the other rules off), then `t_empty` and  $\alpha$  given that best. This cuts the work to a few hundred evaluations.

**Speed trick:** `fast_score` computes macro F0.5 with vectorized pandas and numpy operations (no Python loops over entities), and the pairs are pre-sorted by probability once so each evaluation is quick.

**Tuned on OOF only:** thresholds are chosen on out-of-fold predictions (section 10), never on predictions from models that saw those rows.

| Parameter  | Search range                | Effect of raising it                   |
|------------|-----------------------------|----------------------------------------|
| t_match    | 0.10 to 0.95, step 0.025    | Fewer matches, higher precision        |
| one_to_one | on / off                    | On removes impossible duplicate claims |
| t_empty    | t_match to 0.98, step 0.025 | More empty predictions                 |
| alpha      | 0 to 0.9                    | Drops weaker extra candidates          |

## 12. Engineering concepts

---

A correct algorithm is useless if it runs out of memory or takes a day on GB-size data. These are the engineering ideas that make the pipeline practical, plus the rules it follows.

### Time complexity (Big-O)

**What:** a way to describe how running time grows as data grows.  $O(n)$  means doubling the data doubles the time.  $O(n^2)$  means doubling it quadruples the time.

**Here:** comparing all pairs is  $O(n \times m)$ , which is why blocking exists (section 4). After blocking, the work is roughly  $O(n \times K)$ , linear in the number of S1 records.

### Vectorization

**What:** doing an operation on a whole array at once using optimized C code, instead of looping over items one at a time in Python.

**Why:** a Python loop costs roughly a microsecond or more per item; vectorized numpy or C++ code can be 10–100× faster.

**Here:** normalization uses pandas string operations on whole columns; similarity scores use RapidFuzz `cpdist` (multi-threaded C++, all CPU cores); the metric in `fast_score` uses numpy array math.

### Chunking

**What:** processing data in fixed-size pieces instead of all at once.

**Why:** some operations need memory proportional to their input. Multiplying 1 million S1 vectors by 2 million S2 vectors in one go would need terabytes. In chunks of 2,000 rows, each piece fits in memory, and we keep only the top K results before moving on.

**Here:** sparse similarity search, the postal and rare-token merges, TF-IDF pair cosines, and copying the zip from Drive (64 MB chunks) are all chunked.

### Memory-efficient data types

**What:** storing numbers in the smallest type that fits. A float32 uses half the memory of the default float64; an int8 uses one-eighth of int64.

**Here:** similarity scores are float32, flags are int8, ranks are int16. With millions of rows and 55 features, this can halve memory use, which on Colab decides whether the notebook runs at all.

### Why copy from Google Drive first, and why parquet

**Google Drive in Colab** is mounted over the network. Reading a GB file from it directly is slow and can time out. Copying it once to Colab's local disk makes every later read fast.

**Parquet** is a compressed, column-oriented file format. After the first TSV load, the notebook saves each table as parquet, which reloads in seconds instead of minutes. It also preserves data types exactly.

### Reading TSVs safely

- **dtype=str:** keeps IDs and PIN codes as text. Otherwise "012345" would become the number 12345 and lose its leading zero.

- **keep\_default\_na=False:** keeps empty cells as empty strings. By default pandas turns certain words, like "NA" or "null", into missing values.
- **QUOTE\_NONE:** treats quote marks as normal characters. Otherwise a single stray " in a business name could make pandas merge many lines into one field.
- **Line-count check:** the notebook compares row count with the file's line count to confirm nothing was merged or lost.

## Regex engine consistency

Newer pandas versions store text using Apache Arrow, whose regex engine behaves differently from Python's. The notebook converts text columns to Python objects before regex work, so results are identical on Colab and on newer installations.

## Reproducibility and versioning

**Reproducibility** means anyone can rerun your code and get the same outputs. The challenge requires it for the final package. The notebook fixes random seeds, and saves for every run: both output files, the tuned config, the OOF score, and the five fold models, in a timestamped folder. This is also the submission version history the guidelines ask for.

## Validation of outputs

Before any upload, `self_check` confirms one row per test S1, correct column names, no duplicate IDs, only valid S2/S3 IDs, and every match present in the candidates. Then it runs the official validator. With only 15 submissions in total, a rejected file wastes a scarce resource.

## Compliance with the challenge rules

| Rule                                           | How the pipeline follows it                                                     |
|------------------------------------------------|---------------------------------------------------------------------------------|
| No external data lookup                        | No geocoding, registries or APIs; everything learned from provided files        |
| Model licence MIT/Apache, $\leq 8B$ parameters | LightGBM (MIT), optional MiniLM embeddings (Apache 2.0, $\sim 118M$ parameters) |
| Country is an open set                         | No hard-coded countries; unseen countries fall back to searching all records    |
| candidate_pairs = what the model scores        | The same candidate frame feeds both the model and the file                      |
| Matches $\subseteq$ candidates                 | Guaranteed by construction and checked by <code>self_check</code>               |

## 13. Glossary and interview preparation

### Glossary

| Term                 | One-line meaning                                               | Section |
|----------------------|----------------------------------------------------------------|---------|
| Entity resolution    | Finding records that describe the same real-world thing        | 2       |
| Singleton            | An S1 entity with no true matches                              | 2, 9    |
| Normalization        | Converting text to one standard form                           | 3       |
| NFKD                 | Unicode rule that splits accented letters into letter + accent | 3       |
| Token                | A unit of text, usually a word                                 | 3       |
| Blocking             | Cheap shortlisting of plausible pairs before the model         | 4       |
| Pair recall          | Share of true pairs that survive blocking                      | 4       |
| Reduction ratio      | Share of all possible pairs that blocking removed              | 4       |
| Oracle F0.5          | Score a perfect model would get on our candidates              | 4       |
| Character n-gram     | A sequence of N consecutive letters                            | 5       |
| TF-IDF               | Term weight: frequent here × rare overall                      | 5       |
| Cosine similarity    | How closely two vectors point the same way (0–1)               | 5       |
| Sparse matrix        | Matrix stored by its non-zero entries only                     | 5       |
| Inverted index       | Lookup from each word to the records containing it             | 5       |
| Embedding            | Dense vector encoding a text's meaning                         | 5       |
| FAISS                | Library for fast nearest-neighbour search on vectors           | 5       |
| Levenshtein distance | Minimum single-letter edits to turn one string into another    | 6       |
| Jaro-Winkler         | Name similarity with a bonus for matching prefixes             | 6       |
| Jaccard              | Overlap ÷ union of two sets                                    | 6       |
| Feature engineering  | Designing the numeric inputs a model learns from               | 7       |
| Context feature      | A pair's score relative to its competitors                     | 7       |
| Domain shift         | Test data differing systematically from training data          | 7       |
| Decision tree        | Flowchart of yes/no questions ending in a prediction           | 8       |
| Gradient boosting    | Trees added one by one, each correcting earlier errors         | 8       |
| Log loss             | Error measure that heavily punishes confident mistakes         | 8       |
| Hyperparameter       | A setting chosen before training                               | 8       |
| Regularization       | Any technique that restrains overfitting                       | 8       |
| Early stopping       | Stop training when validation stops improving                  | 8       |
| Hard negative        | A non-match that looks very similar to a match                 | 8       |
| Precision            | Share of predicted matches that are correct                    | 9       |

| Term               | One-line meaning                                            | Section |
|--------------------|-------------------------------------------------------------|---------|
| Recall             | Share of true matches that were found                       | 9       |
| F-beta             | Harmonic blend of precision and recall, weighted by $\beta$ | 9       |
| Macro average      | Score each entity, then average equally                     | 9       |
| AUC                | Probability a true match outranks a non-match               | 9       |
| Cross-validation   | Rotating train/validation splits to measure honestly        | 10      |
| GroupKFold         | Cross-validation that keeps related rows together           | 10      |
| Data leakage       | Validation information accidentally reaching training       | 10      |
| OOF prediction     | A row's prediction from the model that didn't train on it   | 10      |
| LOCO               | Leave-one-country-out validation, simulating France         | 10      |
| Bipartite matching | Pairing items across two sets                               | 11      |
| Grid search        | Trying every combination of settings and keeping the best   | 11      |

## Interview questions with model answers

**Q: Explain your solution in 30 seconds.** It's an entity resolution pipeline. I normalize names and addresses, then use several fast similarity-search blockers to shortlist about 40 candidates per reference record while keeping over 99% of true matches. For each pair I compute about 55 similarity features, score them with LightGBM trained under group-aware cross-validation, then apply decision rules tuned directly for the macro F0.5 metric, including a singleton rule and one-to-one assignment.

**Q: Why do you need blocking at all?** Comparing every pair is quadratic:  $100,000 \times 200,000$  records is 20 billion pairs. Blocking cuts that to a few million, making the model feasible. It's tuned for recall because anything it drops is lost forever.

**Q: Why several blockers instead of one?** Each has blind spots: TF-IDF misses totally renamed businesses, postal blocking misses missing PINs, and so on. Their union covers each other's weaknesses, and I measure each one's individual recall to justify keeping it.

**Q: Why character n-grams?** They tolerate typos, since one typo breaks only a few n-grams, and they need no dictionary, so they work for Hindi transliterations and for French, which isn't in training.

**Q: Why LightGBM and not deep learning or an LLM?** The features are tabular similarity scores, where gradient boosting is the strongest and fastest family. It learns interactions automatically and trains in minutes on CPU. An LLM would be far too slow for millions of pairs and is limited by the challenge's licence and size rules. A fine-tuned cross-encoder would be my next addition, as an extra feature.

**Q: What was your most important feature idea?** Context features. A similarity of 0.8 means little alone; what matters is whether it beats the other candidates. Rank, score gap, reverse rank and mutual-best flags let the model separate near-duplicate businesses.

**Q: How did you avoid overfitting and leakage?** GroupKFold by S1 entity, because rows of one entity share context features and must stay together. Early stopping. Threshold tuning only on out-of-fold predictions. And leave-one-country-out validation to test generalization.

**Q: How did you handle France with zero training examples?** Accent-stripping Unicode normalization, language-agnostic character n-grams, no country feature so the model can't learn country-specific rules, LOCO validation to simulate an unseen country, and slightly stricter thresholds for unseen countries.

**Q: Why isn't your decision just probability > 0.5?** 0.5 optimizes accuracy. The metric is precision-weighted, averaged per entity, and gives 1.0 or 0.0 on singletons. So I grid-searched a match threshold, a separate singleton threshold, a relative cutoff, and one-to-one assignment directly on macro F0.5.

**Q: Why is one-to-one assignment valid?** Source 1 is deduplicated, so each S2/S3 record describes a business that appears once in S1. It can match at most one S1 entity. I verified this on the training labels, then kept each record only for its highest-scoring S1. The Hungarian algorithm would wrongly force one-to-one on the S1 side too.

**Q: Why not use accuracy or AUC to evaluate?** Accuracy is inflated by the huge number of easy non-matches. AUC ignores the threshold. I use AUC only to monitor training, and judge the system by the official macro F0.5.

**Q: What are the limitations and what would you improve?** Blocking still caps recall. The unseen-country margin is a judgment call. Next I'd add a cross-encoder score as a feature, a second-stage model using the first model's ranks, per-source thresholds, and a systematic error-analysis loop on false positives and missed pairs.

**Q: How did you make it scale to GB-size data?** Chunked sparse matrix multiplication for similarity search, multi-threaded C++ string similarity, compact data types, parquet caching, and copying data from Drive to local disk before reading.

## Numbers to fill in from your real run

Interviewers find concrete numbers convincing. Record these after your final run:

| Measure                                                  | Your value |
|----------------------------------------------------------|------------|
| Pair recall after blocking                               |            |
| Mean candidates per S1                                   |            |
| Reduction ratio                                          |            |
| Oracle F0.5                                              |            |
| OOF macro F0.5                                           |            |
| LOCO F0.5 (US held out / India held out)                 |            |
| Tuned $t_{\text{match}}$ / $t_{\text{empty}}$ / $\alpha$ |            |
| Public and private leaderboard score                     |            |
| Top 5 features by gain                                   |            |